

前言
01.学习概述
1.1 学习目标
1.2 前置知识
02.核心概念
2.1 需求场景
2.2 实现方案
2.3 技术选型
03.整体流程
3.1 架构设计
3.2 代码执行流
04. 核心代码讲解
4.1 ETS与Native通信机制
4.2 ftools封装与处理 (重点)
4.3 CMake构建配置
05.跨语言通信原理 (虚拟机语言 ↔ Native)
5.1 为什么需要跨语言通信机制
5.2 核心机制
5.3 完整调用流程
5.4 aki的价值
05.深度思考
5.1 关键问题探究
5.2 设计对比
06.实践验证
6.1 行为验证代码
6.2 性能测试
07.应用场景
7.1 最佳实践
7.2 使用禁忌
08.总结提炼
8.1 核心收获
8.2 知识图谱
8.3 延伸思考
09.参考资料
其他介绍
01.关于我的博客

前言

学习要符合如下的标准化链条：了解概念->探究原理->深入思考->总结提炼->底层实现->延伸应用"

01.学习概述

- 学习主题：
- 知识类型：
 - ☐ ☒Android/
 - ☐ ☒01.基础组件与机制
 - ☐ ☒四大组件
 - ☐ ☒IPC机制

- ☐ ☒ 消息机制
- ☐ ☒ 事件分发机制
- ☐ ☒ View与渲染体系 (含Window、复杂控件、动画)
- ☐ ☒ 存储与数据安全 (SharedPreferences/DataStore/Room/Scoped Storage)
- ☐ ☒ 02. 架构与工程化
 - ☐ ☒ 架构模式 (MVC/MVP/MVVM/MVI)
 - ☐ ☒ 依赖注入 (Koin/Hilt/Dagger)
 - ☐ ☒ 路由与模块化 (ARouter、Navigation)
 - ☐ ☒ Gradle与构建优化
 - ☐ ☒ 插件化与动态化
 - ☐ ☒ 插桩与监控框架
- ☐ ☒ 03.性能优化与故障诊断
 - ☐ ☒ ANR分析与优化
 - ☐ ☒ 启动耗时优化
 - ☐ ☒ 内存泄漏监控
 - ☐ ☒ 监控与诊断工具
- ☐ ☒ 04.Jetpack与生态框架
 - ☐ ☒ Room
 - ☐ ☒ Paging
 - ☐ ☒ WorkManager
 - ☐ ☒ Compose
- ☐ ☒ 05.Framework与系统机制
 - ☐ ☒ ActivityManagerService (含ANR触发机制)
 - ☐ ☒ Binder机制
- ☐ ☒ 音视频开发/
 - ☐ ☒ 01.基础知识
 - ☐ ☒ 02.OpenGL渲染视频
 - ☐ ☒ 03.FFmpeg音视频解码
- ☐ ☒ Java/
 - ☐ ☒ 01.基础知识
 - ☐ ☒ 02.集合框架
 - ☐ ☒ 03.异常处理
 - ☐ ☒ 04.多线程与并发
 - ☐ ☒ 06.JVM
- ☐ ☒ Kotlin/
 - ☐ ☒ 01.基础语法
 - ☐ ☒ 02.高阶扩展
 - ☐ ☒ 03.协程和流
- ☐ ☒ Flutter/

- ☐ ☒ 01.基础知识
 - ☐ ☒ Dart 语言基础
 - ☐ ☒ Widget 基础与生命周期
 - ☐ ☒ Flutter 基础组件
 - ☐ ☒ 布局与约束
 - ☐ ☒ 绘制与渲染体系
 - ☐ ☒ 状态管理
 - ☐ ☒ 事件处理与手势系统
 - ☐ ☒ 原生通信
- ☐ ☒ 02.路由与导航
- ☐ ☒ 03.性能优化与故障诊断
- ☐ ☒ 04.异步编程
- ☐ ☒ 05.项目经验与案例沉淀
- ☐ ☒ 自我管理/
 - ☐ ☒ 01.内观
- ☐ ☒ 项目经验/
 - ☐ ☒ 01.启动逻辑
 - ☐ ☒ 02.云值守
 - ☐ ☒ 03.智控平台
 - ☐ ☒ 04.视频巡店

- 学习来源:
- 重要程度: ★★★★★
- 学习日期: 2025.
- 记录人: @panruiqi

1.1 学习目标

- 了解概念->探究原理->深入思考->总结提炼->底层实现->延伸应用"

1.2 前置知识



02.核心概念

2.1 需求场景

录像下载，要求下载对应url的录像，并转换为mp4格式

比如：有一个URL: https://sns-video-al.xhscdn.com/stream/110/405/01e583cb6e0fed5a010370038c8ad962fb_405.flv.

我们进行录像下载，下载后还要转换为mp4格式输出

2.2 实现方案

将FFmpeg命令行工具（fftools）封装成可供ArkTS调用的Native库。然后通过FFmpeg命令去控制ffmpeg.so执行相关任务

- 类似：
 - ffmpeg -i https://sns-video-al.xhscdn.com/stream/110/405/01e583cb6e0fed5a010370038c8ad962fb_405.avi -c:v copy -c:a copy -f avi -y /data/storage/el2/base/haps/entry/files/test_output.mp4

其实本质就是：通过命令行调用ffmpeg.so执行相关任务，就这么简单。

本质很简单，但是实现很困难，涉及到多个部分：

1. ArkTs如何与C层通信？
2. ffmpeg源码本身不为鸿蒙设计，其异常时会直接退出应用进程，我们要修改他，让他可以满足我们app的需求
3. 需要编译鸿蒙侧的FFmpeg.so，并补全其依赖库，如：rtmpdump等。为上层提供基础能力，这涉及到CMakeList和交叉编译（还要开启一个fPIC）
4. ffmpeg源码依赖很多.h，我们要有引入补全他们。

整体而言，上层反而是最容易的，难点在于native和原生通信，ffmpeg.so的交叉编译，以及ffmpeg工具的修改引入

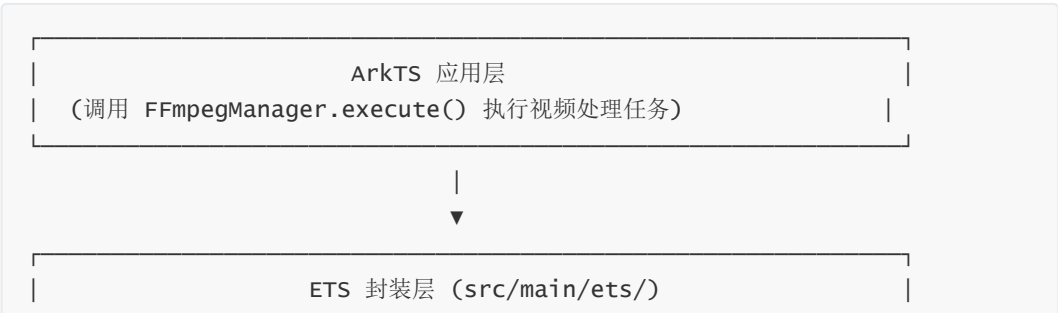
2.3 技术选型

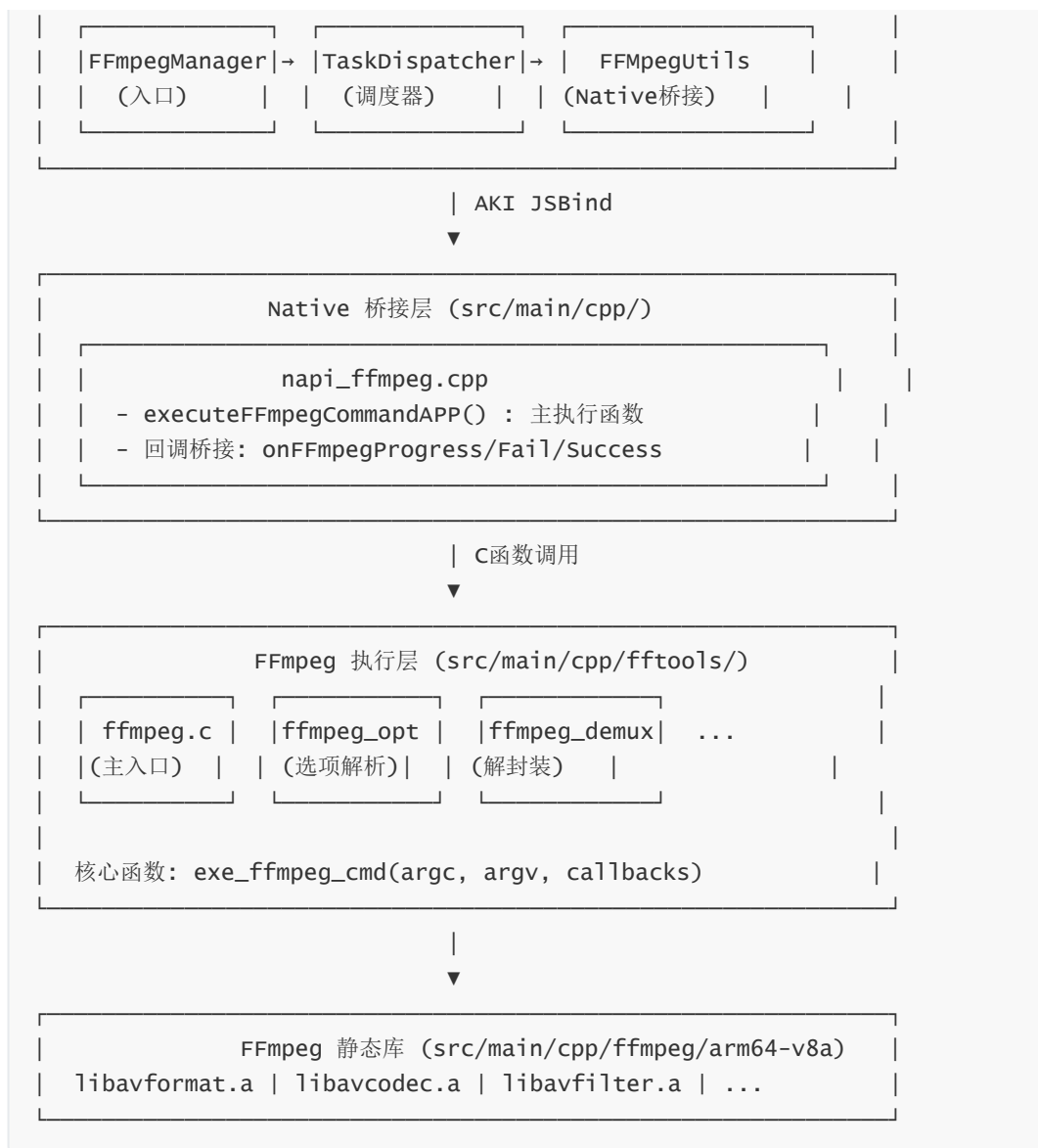
技术	用途
AKI框架 (@ohos/aki)	ArkTS与c层通信，简化NAPI开发
FFmpeg静态库	预编译的arm64-v8a架构FFmpeg库，提供ffmpeg核心能力
fftools源码	FFmpeg命令行工具源码，手动修改内部实现以支持鸿蒙侧进行调用

03.整体流程

3.1 架构设计

- 整体如下：
 - ARKTS应用层：其他lib通过调用当前lib_FFmpeg_Tools执行相关功能
 - ETS封装层：其就是当前lib_FFmpeg_Tools，会将其他lib的调用封装成一个ffmpegtools工具的cmd命令。并通过任务池管理和进行调度执行，最终执行是通过委托模式通过AKI JSBind委托给native层处理
 - AKI接受到任务后会在自身线程池中启动线程执行 napi_ffmpeg.cpp 的 executeFFmpegCommandAPP；该命令会调用到fftools工具
 - ffmpeg调用到底层的ffmpeg的静态库：libavcodec.a；libavfilter.a等
 - 至此整体调用流程完成





- 这里的native和ets通信全是依赖aki实现的 (ets调用native 和 native回调ets)

3.2 代码执行流

- 看看代码的执行流





```
| FFMpegUtils.executeFFmpegCommand() 【关键：ETS→Native桥梁】
|
| └─ 1. 生成32位UUID
|
| └─ 2. 通过AKI绑定回调函数到全局：
|
| | libAddon.JSBind.bindFunction(uuid+"_onFFmpegProgress",
callback) |
| | libAddon.JSBind.bindFunction(uuid+"_onFFmpegFail", callback)
|
| | libAddon.JSBind.bindFunction(uuid+"_onFFmpegSuccess",
callback) |
| └─ 3. 调用Native: libAddon.executeFFmpegCommandAPP(uuid, cmdLen,
cmds) |
```

| AKI JSBind



```
| napi_ffmpeg.cpp :: executeFFmpegCommandAPP() 【Native层】
|
| └─ 1. 转换参数: vector<string> → char**
|
| └─ 2. 获取ArkTS回调函数指针：
|
| | aki::JSBind::GetJSFunction(uuid + "_onFFmpegProgress")
|
| └─ 3. 构建C回调结构体 callbacks
|
| └─ 4. 调用 exe_ffmpeg_cmd(argc, argv, &callbacks)
```

| C函数调用



```
| ffmpeg.c :: exe_ffmpeg_cmd() 【FFmpeg执行层】
|
| └─ 解析命令行参数
|
| └─ 打开输入文件/流
|
| └─ 创建输出文件
|
| └─ 转码/复制流
|
| └─ 执行过程中调用 callbacks->onFFmpegProgress(progress)
|
| └─ 完成后返回结果码
```

04. 核心代码讲解

难点在于native和原生通信，ffmpeg.so的交叉编译，以及fftools工具的修改引入

但是就代码而言，目前能看的只有native层与原生的通信机制，以及fftools工具的修改

4.1 ETS与Native通信机制

AKI框架：

- 作用：简化了napi，提供相关封装，我们通过对其调用可以快速实现和native层通信

原生调用native流程

- Native层注册函数

```
o // napi_ffmpeg.cpp

// 1. 注册模块名
JSBIND_ADDON(ffmpegutils) // 对应 libffmpegutils.so

// 2. 注册全局函数
JSBIND_GLOBAL() {
    JSBIND_PFUNCTION(executeFFmpegCommandAPP); // 异步
    JSBIND_FUNCTION(showLog); // 同步
}
- `JSBIND_PFUNCTION` 让函数在AKI线程池中异步执行，返回Promise
- `JSBIND_FUNCTION` 在JS主线程同步执行
```

- ETS层调用

```
o // FFmpegUtils.ets
import libAddon from 'libffmpegutils.so' // 加载.so库

// 直接调用Native函数（AKI自动映射）
libAddon.executeFFmpegCommandAPP(uuid, cmdLen, cmds);
libAddon.showLog(true);
```

- 此时，通过函数名的映射，我们很自然的调用到了so中的c代码执行位置

native层回调原生

- 如果是Android，那么我们要获得JVM虚拟机和env。然后保存方法ID，通过方法ID找到jMethod，并执行其static方法，也就是JNI机制
- ETS层：注册回调
 - o 我们直接将回调函数绑定到全局，然后native层只传递uuid

- ```
// FFMpegUtils.ets
let uuid = FFMpegUtils.generateUUID32(); // 生成唯一标识

// 将回调函数绑定到全局，以 uuid_函数名 命名
libAddon.JSBind.bindFunction(uuid + "_onFFmpegProgress",
options.onFFmpegProgress);
libAddon.JSBind.bindFunction(uuid + "_onFFmpegFail",
options.onFFmpegFail);
libAddon.JSBind.bindFunction(uuid + "_onFFmpegSuccess",
options.onFFmpegSuccess);

// 调用Native，传递uuid
libAddon.executeFFmpegCommandAPP(uuid, options.cmds.length,
options.cmds);
```

- Native层：获取回调

- ```
// napi_ffmpeg.cpp
int executeFFmpegCommandAPP(std::string uuid, int cmdLen,
std::vector<std::string> argv) {
    // 通过uuid找到对应的JS回调函数
    CallbackInfo onActionListener;
    onActionListener.onFFmpegProgress = aki::JSBind::GetJSFunction(uuid
+ "_onFFmpegProgress");
    onActionListener.onFFmpegFail = aki::JSBind::GetJSFunction(uuid +
"_onFFmpegFail");
    onActionListener.onFFmpegSuccess = aki::JSBind::GetJSFunction(uuid
+ "_onFFmpegSuccess");

    // 调用JS回调
    onActionListener.onFFmpegProgress->Invoke<void>(progress);
}
```

- 看起来很简单，通过名称从aki中找到js回调函数

关键是aki的机制，我们通过上面的api就可以实现ets层和native的通信。那么原理呢？其实这就是跨语言通信的过程，我们后面5.跨语言通信原理（虚拟机语言 ↔ Native）中说

4.2 ffmpeg封装与处理（重点）

问题：原生FFmpeg的 `exit_program()` 会调用 `exit()` 直接终止进程，这在库调用场景下是不可接受的，因为会导致我们的应用进程崩溃。

解决方案：使用 `setjmp/longjmp` 实现非局部跳转，优雅返回错误码。

- 原代码（ffmpeg原版）：

- ```
void exit_program(int ret) {
 if (program_exit)
 program_exit(ret);
 exit(ret); // ✗ 直接退出进程
}
```

- 修改后（cmdutils.c）：

- `// 线程局部存储的返回值`  
`__thread volatile int longjmp_value;`  
  
`void exit_program(int ret) {`  
 `if (program_exit)`  
 `program_exit(ret);`  
  
 `// 保存返回值并跳转`  
 `longjmp_value = ret;`  
 `longjmp(ex_buf__, ret); // 跳转到setjmp处`  
`}`

- 我们在哪setjmp呢?
- 在exe\_ffmpeg\_cmd中setjmp, 根据返回值进行相关处理, 并最终return执行的结果码

- `int exe_ffmpeg_cmd(int argc, char **argv, Callbacks *callbacks) {`  
 `int ret;`  
  
 `// 保存回调指针`  
 `g_callbacks = callbacks;`  
  
 `// 设置跳转点`  
 `int savedCode = setjmp(ex_buf__);`  
  
 `if (savedCode == 0) {`  
 `// ===== 正常执行流程 =====`  
 `init_dynload();`  
 `register_exit(ffmpeg_cleanup);`  
 `avformat_network_init();`  
  
 `ret = ffmpeg_parse_options(argc, argv); // 解析参数`  
 `if (ret < 0)`  
 `exit_program(1); // 会跳转到else分支`  
  
 `if (transcode() < 0) // 执行转码`  
 `exit_program(1);`  
  
 `exit_program(received_nb_signals ? 255 : main_return_code);`  
  
 `} else {`  
 `// ===== longjmp跳转后到达这里 =====`  
 `main_return_code = (received_nb_signals) ? 255 : longjmp_value;`  
 `}`  
  
 `return main_return_code; // 优雅返回, 不会exit`  
`}`

## 4.3 CMake构建配置

- 如下
  - 将native层一并构建为一个ffmpegtools.so

- `# CMakeLists.txt 关键配置`  
  
`# FFmpeg静态库链接顺序（很重要！）`  
`target_link_libraries(ffmpegutils PRIVATE`

```

 ${FFMPEG_LIB}/libavdevice.a
 ${FFMPEG_LIB}/libavfilter.a
 ${FFMPEG_LIB}/libavformat.a
 ${FFMPEG_LIB}/libavcodec.a
 ${FFMPEG_LIB}/libswresample.a
 ${FFMPEG_LIB}/libswscale.a
 ${FFMPEG_LIB}/libavutil.a

 # 依赖库
 ${FFMPEG_LIB}/librtmp.a
 ${FFMPEG_LIB}/libssl.a
 ${FFMPEG_LIB}/libcrypto.a

 # 鸿蒙系统库
 libace_napi.z.so
 libhilog_ndk.z.so

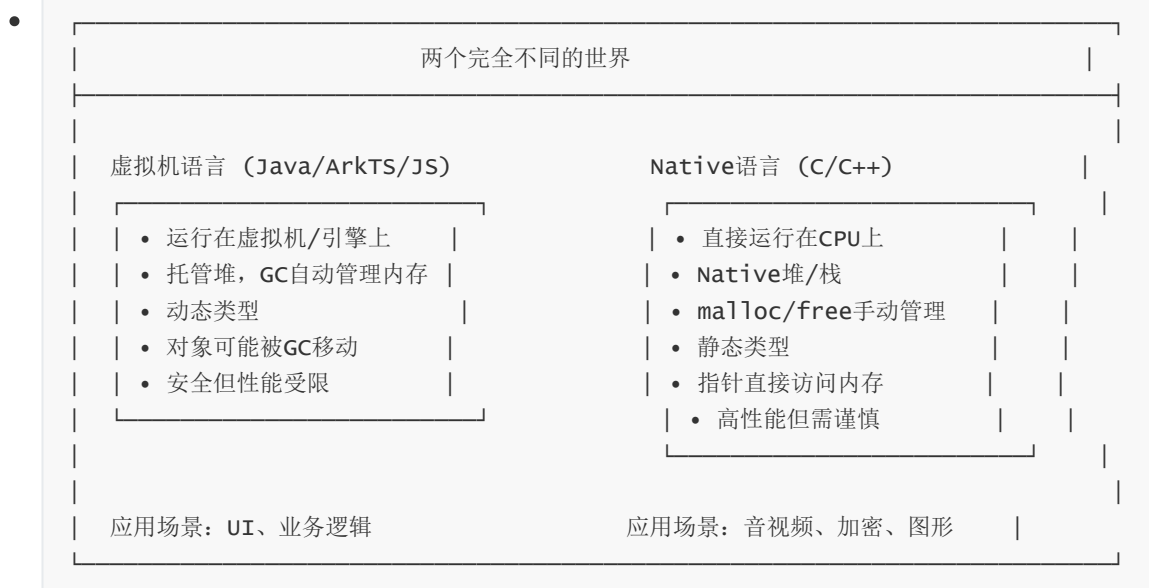
 # AKI框架
 Aki::libjsbind
)

```

## 05.跨语言通信原理（虚拟机语言 ↔ Native）

### 5.1 为什么需要跨语言通信机制

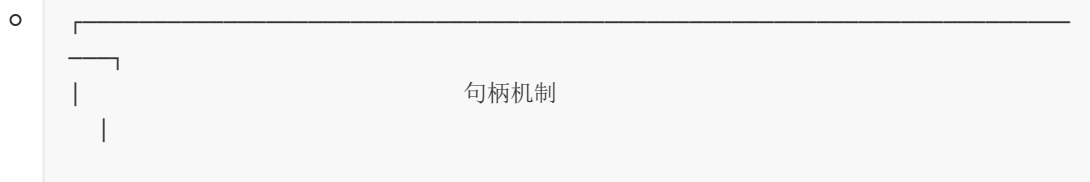
跨语言通信的难点在于：两者内存模型完全不同，无法直接互操作。



### 5.2 核心机制

句柄（Handle）—— 间接访问

- 问题：GC会移动对象，Native不能直接持有虚拟机堆地址。
- 解决：通过句柄间接访问。





```

|
|
| 两份独立的数据，互不影响
|
| - 修改buf不会影响JSString
|
| - GC回收JSString不会影响buf
|
|_____
|_____

```

- native传递数据到虚拟机则是通过创建虚拟机中的对象，然后把句柄传给原生层

```

○
|_____
|_____
| 返回值是"创建"，不是"写入"
|_____
|_____
|
|
| Native执行完毕：
|
| int result = 42;
|
| |
|
| ▼ napi_create_int32(env, 42, &js_result)
|
|
|
| 引擎内部：
|
| 1. 在虚拟机堆中分配空间
|
| 2. 创建JSNumber对象，值为42
|
| 3. 返回句柄给Native
|
| |
|
| ▼
|
| Native拿到句柄，return给JS层
|_____
|_____

```

## 引用管理 —— 防止GC回收

- 引用类型

临时句柄 (Local Reference)

- 函数调用期间有效
- 函数返回后自动失效
- 适用于：参数、临时变量

持久引用 (Global Reference / napi\_ref)

- 手动创建，手动释放
- 告诉GC：这个对象被Native持有，不要回收
- 适用于：回调函数、长期缓存的对象

创建：napi\_create\_reference(env, obj, 1, &ref)

使用：napi\_get\_reference\_value(env, ref, &obj)

释放：napi\_delete\_reference(env, ref)

## 5.3 完整调用流程

虚拟机 → Native

- 如下

JS/ArkTS 调用 Native 函数

Step 1: JS发起调用

libAddon.executeCommand("hello", 123, [1,2,3])



Step 2: 引擎准备调用

- 查找Native函数指针
- 将JS参数转为句柄数组：[handle1, handle2, handle3]
- 创建 napi\_callback\_info

▼

Step 3: Native函数执行

```
napi_value my_func(napi_env env, napi_callback_info info) {
 // 获取参数句柄

 napi_get_cb_info(env, info, &argc, argv, ...);

 // 通过句柄复制数据到Native

 napi_get_value_string_utf8(env, argv[0], buf, ...);

 // 现在 buf = "hello"

 // 执行C逻辑...

 int result = do_something(buf);

 // 创建返回值（在虚拟机堆中创建新对象）
 napi_value js_result;

 napi_create_int32(env, result, &js_result);

 return js_result;
}
```

▼

Step 4: 返回JS层

JS拿到返回值，继续执行

Native → 虚拟机（回调）

- 如下

o

## Native 调用 JS 回调函数

### Step 1: JS注册回调

```
libAddon.bindCallback("onProgress", (progress) => {
 console.log(progress);
});
```



### Step 2: Native保存引用

```
// 创建持久引用，防止GC回收
napi_create_reference(env, js_callback, 1, &g_callback_ref);
```



### Step 3: Native需要回调时

```
// 从引用获取函数句柄
napi_value callback;
napi_get_reference_value(env, g_callback_ref, &callback);

// 准备参数（创建JS对象）
napi_value args[1];
napi_create_int32(env, 50, &args[0]); // progress = 50
```





```
| | napi_get_value_string_utf8(env, argv[0], buf, len+1, &len);|
|
| | // ... 更多转换代码 ... |
|
| | } |
|
|
|
|
| 使用AKI（约5行）：
|
|
| | int myFunc(std::string str, int num) { |
|
| | // 直接使用C++类型，AKI自动转换 | |
| | return 0; |
|
| | } |
|
| | JSBIND_FUNCTION(myFunc); |
|
|
|
|
|
```

## 05.深度思考

---

### 5.1 关键问题探究

### 5.2 设计对比

## 06.实践验证

---

### 6.1 行为验证代码

### 6.2 性能测试

## 07.应用场景

---

### 7.1 最佳实践

## 7.2 使用禁忌

# 08.总结提炼

---

## 8.1 核心收获

## 8.2 知识图谱

## 8.3 延伸思考

# 09.参考资料

---

- 1.
- 2.
- 3.

# 其他介绍

---

## 01.关于我的博客

- csdn: [http://my.csdn.net/qg\\_35829566](http://my.csdn.net/qg_35829566)
- 掘金: <https://juejin.im/user/499639464759898>
- github: <https://github.com/jjjjjjava>
- 邮箱: [[934137388@qq.com](mailto:934137388@qq.com)]